

EECS1015 – Fall 2022 – Practice Final Exam (Lab 9)

(c) Michael S. Brown -- This document may not be shared or reused without permission from the author.

INSTRUCTIONS

1. I'm writing this lab as if it were a final exam; however, please submit it to lab 9.
2. You have 90 minutes (1.5 hours) to complete the exam.
3. There are 100 points possible. Questions are worth different amounts.
4. Please have your photo ID out and all other items put away.
5. When you finish the exam, upload your python code to websubmit (see below).
6. **IMPORTANT: Do not log out of your PC when you finish. Leave your code on the screen. The TAs and I will look at your code immediately following the exam.**

GETTING STARTED

1. Log on to the PC.
2. On the browser, you will see the following:
 - (a) Python "cheat sheet" (PDF)
 - (b) Final exam (PDF)
 - (c) Final starter code (text file)
 - (d) Video showing each task running (no audio)
 - (e) Link to the websubmit
2. Open Pycharm (wait for it to initialize)
 - (a) Create a new project
 - (b) Copy and paste the starter code into your main.py.

TASK0 [0 points, -10 points if you forget]

1. Complete the information in Task0 given in the main.py file.

SUBMITTING YOUR FINAL TO WEB SUBMIT

1. When you finish your exam, please upload it at this link:
<https://webapp.eecs.yorku.ca/submit> (you can also click the link on the exam webpage)
2. Select your section A or B
3. Select "Final" (well, Lab 9 for the practice)
4. Upload from your PyCharmProject folder the "main.py". (You don't need to rename it to final.py as shown below, main.py is OK).

The screenshot shows the websubmit web application interface. At the top, there are four dropdown menus for selection: 'Academic Year' (set to 2022-23), 'Term' (set to F), 'Course' (set to 1015A), and 'Assignment' (set to Lab 9). Below these, the 'Submit Status' is shown as 'Submission' and 'Enabled'. The 'Feedback' is shown as 'None'. A section titled 'Please specify files to submit: (You can submit multiple files at once!)' contains a table with file upload options. The first row shows a 'Choose Files' button next to a text box containing 'lab9.py'. The subsequent six rows show 'Choose Files' buttons next to text boxes that say 'No file chosen'.

Academic Year:	2022-23 ▼
Term:	F ▼
Course:	1015A ▼
Assignment:	Lab 9 ▼
Submit Status: Submission	
Enabled	
Feedback: None	
Please specify files to submit: (You can submit multiple files at once!)	
Choose Files	lab9.py
Choose Files	No file chosen
Choose Files	No file chosen
Choose Files	No file chosen
Choose Files	No file chosen
Choose Files	No file chosen
Choose Files	No file chosen

Task 1 [15pts] - Removing words containing substrings

Topics: *list processing, string split, and join*

This task will ask the user to input a long string containing words and a short substring. From this input, generate two new strings: (1) consisting of words containing the substring; and (2) one without (W/O) words containing the substring.

Task 1 should work as follows.

1. Ask the user to input a long sentence.
2. Ask the user to input a substring (see example below).
3. Print out the sentence (surrounded by single quotes) with the words containing the substring entered.
4. Print out the sentence (surrounded by single quotes) without the words containing the substring entered
5. Ask the user if they want to repeat this task; if "Y"/"y" is entered, go back to 1.

Sample outputs (input in *red*)

--- Task 1 ---

Type in a long sentence: *This is a long sentence.*

Remove words containing: *is*

With substring: 'This is'

W/O substring: 'a long sentence.'

Try again? [Y/N] *y*

Type in a long sentence: *The brown fox jumped over the log.*

Remove words containing: *o*

With substring: 'brown fox over log.'

W/o substring: 'The jumped the'

Try again? [Y/N] *Y*

Type in a long sentence: *How much wood would a woodchuck chuck, if a woodchuck could chuck wood.*

Remove words containing: *wood*

With substring: 'wood woodchuck woodchuck wood.'

W/o substring: 'How much would a chuck, if a could chuck'

Try again? [Y/N] *N*

Print outputs surrounded by single quotes.

Task 2 [25pts] - List reshaping

Topics: *lists, nested collections, functions*

This task will ask the user to input a number N. The task will generate a list of N random numbers ranging from 0-9. The task will then "reshape" the list of N numbers to fit within a *list of lists* based on a specified number of rows and cols.

You need to define the following functions:

`randomlist(param: N)` -> returns a list

param:

N is the number of random numbers to put in the list

- Your function should generate N random numbers between 0-9 and insert them in a list.
- Return the list.

`reshape(params: a_list, num_rows, num_cols)` -> returns a list of lists

params:

a_list - a list of numbers.

num_rows, num_cols - number of rows and cols.

- This function should create a new list containing num_rows lists.
- Each "row" list will have num_cols items.
- Loop through a_list and assign each item to the "new list," filling your new list of lists row by row (see example below).
- Return the new lists of lists.

Task 2 should work as follows.

1. Ask the user to input a number N (you may assume it will be positive).
2. Call function `randomlist()` to generate a random list of N numbers. Print the random list.
4. Ask the user to input rows and then cols.
5. If (rows*cols) is not equal to N, print an error and then go back to 4.
4. Otherwise, call function `reshape()` with the randomlist and user-provided rows, cols.
5. Print the list returned by function `reshape()`.
6. Ask the user if they want to repeat the task; if "Y" or "y" go back to 1.

Sample outputs (input in red)

--- Task 2 ---

List length: 15

[0, 8, 3, 1, 7, 4, 1, 4, 9, 8, 7, 3, 1, 2, 2]

Rows: 5

Cols: 3

Reshaped List

[[0, 8, 3], [1, 7, 4], [1, 4, 9], [8, 7, 3], [1, 2, 2]]

Try again? [Y/N] y

List length: 15

[5, 4, 6, 8, 2, 2, 5, 4, 5, 2, 0, 7, 8, 1, 4]

Rows: 3

Cols: 9

Error: 3*9 != 15

Rows: 3

Cols: 5

Reshaped List

[[5, 4, 6, 8, 2], [2, 5, 4, 5, 2], [0, 7, 8, 1, 4]]

Try again? [Y/N] y

List length: 18

[7, 3, 0, 3, 6, 6, 4, 3, 7, 3, 6, 8, 8, 7, 1, 0, 5, 6]

Rows: 9

Cols: 2

Reshaped List

[[7, 3], [0, 3], [6, 6], [4, 3], [7, 3], [6, 8], [8, 7], [1, 0], [5, 6]]

Try again? [Y/N] n

Ex1: "Reshaped" list has 5 lists (i.e., 5 rows), each with 3 items (i.e., 3 columns) **[total items 15]**.

Example of invalid input for rows and cols. Ask again.

Ex2: "Reshaped" list has 3 lists (i.e., 3 rows), each with 5 items (i.e., 5 columns) **[total items 15]**.

Ex3: "Reshaped" list has 9 lists (i.e., 9 rows), each with 2 items (i.e., 2 columns) **[total items 18]**.

Task 3 [30pnts] - Finding keys with duplicate values in a dictionary

Topics: [dictionaries](#), [functions](#)

This task will ask the user to input a series of words. These words will be inserted into a dictionary where the key is the order that the word was inputted (e.g., 1, 2, 3); the value is the word. You will then call a function that will create a new dictionary of duplicate words, where the word is the key, and the value is a list, where each item in the list is the order (key from the other dictionary) in which the word appeared. See the output below.

You need to define the following function:

`find_duplicates(param: a_dict)` -> returns a dictionary

`param: a_dict` is a dictionary in the following form:

```
{1:"word1", 2:"word2", 3:"word1", 4:"word2", ..}
```

Your function should find the keys that have the same values (i.e., duplicates).

Add these duplicates to a new dictionary where the key is the duplicated word, and the value is a list of all keys that have the word as its value. For example, processing the dictionary above:

```
{"word1":[1, 3], "word2":[2, 4], ..}
```

Return the newly created dictionary.

Task 3 should work as follows:

1. Ask the user to input a word (and enter an empty string to the end).
2. Each time the user inputs a word add it to a dictionary where the key is the # of input (start with 1) and the value is the word. See formatting below for the input prompt below.
3. When an empty string is entered (i.e., the user press enter only), call function `find_duplicates()`, passing your word dictionary as the parameter.
4. Print out the dictionary returned by `find_duplicates()`.
5. Ask the user if they'd like to repeat the task; if "Y" or "y", go back to 1.

Sample outputs (input in red)

Input words, press enter to end.

[Input 1] Word: **This**

[Input 2] Word: **is**

[Input 3] Word: **only**

[Input 4] Word: **a**

[Input 5] Word: **test**

[Input 6] Word: **This**

[Input 7] Word: **test**

[Input 8] Word: **has**

[Input 9] Word: **always**

[Input 10] Word: **been**

[Input 11] Word: **a**

[Input 12] Word: **test**

[Input 13] Word: **(enter)**

Dictionary

```
{1: 'This', 2: 'is', 3: 'only', 4: 'a', 5: 'test', 6: 'This', 7: 'test', 8: 'has', 9: 'always', 10: 'been', 11: 'a', 12: 'test'}
```

Duplicates

```
{'This': [1, 6], 'a': [4, 11], 'test': [5, 7, 12]}
```

Try again? **Y**

Input words, press enter to end.

[Input 1] Word: **No**

[Input 2] Word: **repeats**

[Input 3] Word: **here.**

[Input 4] Word: **(enter)**

Dictionary

```
{1: 'No', 2: 'repeats', 3: 'here.'}
```

Duplicates

```
{}
```

Try again? **N**

Input "count" is printed with width 2.

Dictionary based on input. Input #s are the key, inputs are the value.

Repeated words are the key, value are a list of the input #.

If no repeats, `find_duplicates()` should return an empty dictionary.

Task 4 [30pts]: Range checker object

Topics: `Lists`, `split`, `classes`, `objects`

For this task you need to define a class called "rangeChecker" that is initialized with a string (name) and two numbers (min and max). This object can check if a number is either within range or outside range.

You need to define the following class:

Class name: `rangeChecker`

Class data (1)

`range_counter` An integer that keeps track of how many ticket objects have been created. Initialize to 1.

Instance data (4)

`name` The name of our range checker.
`id` The id of our range checker.
`min_value` The minimum value of our range checker.
`max_value` The maximum value of our range checker.

Methods (3)

`__init__(name, min, max)`

- Use `assert` to ensure that `max > min`, if not print `AssertionError` as shown below.
- Set instance variable `id` to the class variable `range_counter` and add 1 to `range_counter`.
- Assign parameters `name`, `max`, and `min` to instance variables `name`, `min_value`, `max_value`.

`within_range(number)` -> Bool

- Return True if `min_value ≤ number ≤ max_value`
- Return False otherwise

`outside_range(number)` -> Bool

- Return True if `number < min_value` or `number > max_value`
- Return False otherwise

`print()` -> no return

- Print `rangeChecker`'s `name` (width 10), `id` (width 2) and `min/max_values` (width 8, precision 2)
- See example below

Task 4 should work as follows:

1. You should ask the user to input three `rangeChecker` objects.
2. Each line of input ask for a name, a min value (float) and max value (float).
3. Create a `rangeChecker` object based on the entered information.
4. Now ask the user to input a list of numbers (floats).
5. Loop through each of three `rangeChecker` objects.
6. Call the `rangeChecker` object's `print()` method to bring out its id, name, and ranges.
7. For each number the user inputted, call the `rangeChecker` object methods `within_range()` and `outside_range()`. (Follow formatting shown in example output).
8. Ask the user if they want to repeat the task, if "Y" or "y", got back to 1.

Output shown on next pages.

Sample outputs (input in red)

--- Task 4 ---

Range 0 Name, Min, Max: ones, 0, 1

Range 1 Name, Min, Max: tens, 0, 10

Range 2 Name, Min, Max: hundreds, 0, 100

Input list of numbers x1,x2,...,xn: 0.5, 99, 5, 11

rangeChecker [1] 'ones' - 0.00 <= num <= 1.00

Inside range [0.50]: True

Inside range [99.00]: False

Inside range [5.00]: False

Inside range [11.00]: False

rangeChecker [2] 'tens' - 0.00 <= num <= 10.00

Inside range [0.50]: True

Inside range [99.00]: False

Inside range [5.00]: True

Inside range [11.00]: False

rangeChecker [3] 'hundreds' - 0.00 <= num <= 100.00

Inside range [0.50]: True

Inside range [99.00]: True

Inside range [5.00]: True

Inside range [11.00]: True

rangeChecker [1] 'ones' - 0.00 <= num <= 1.00

Outside range [0.50]: False

Outside range [99.00]: True

Outside range [5.00]: True

Outside range [11.00]: True

rangeChecker [2] 'tens' - 0.00 <= num <= 10.00

Outside range [0.50]: False

Outside range [99.00]: True

Outside range [5.00]: False

Outside range [11.00]: True

rangeChecker [3] 'hundreds' - 0.00 <= num <= 100.00

Outside range [0.50]: False

Outside range [99.00]: False

Outside range [5.00]: False

Outside range [11.00]: False

Try again? y

Range 0 Name, Min, Max: target1, 0.5, 1.5

Range 1 Name, Min, Max: target2, 2.2, 3.4

Range 2 Name, Min, Max: target3, -2.0, -0.5

Input list of numbers x1,x2,...,xn: 0.1, -0.8, 2.5, 1

rangeChecker [4] 'target1' - 0.50 <= num <= 1.50

Inside range [0.10]: False

Inside range [-0.80]: False

Inside range [2.50]: False

Inside range [1.00]: True

rangeChecker [5] 'target2' - 2.20 <= num <= 3.40

Inside range [0.10]: False

Inside range [-0.80]: False

Inside range [2.50]: True

Inside range [1.00]: False

rangeChecker [6] 'target3' - -2.00 <= num <= -0.50

Inside range [0.10]: False

Inside range [-0.80]: True

Inside range [2.50]: False

Inside range [1.00]: False

rangeChecker [4] 'target1' - 0.50 <= num <= 1.50

Outside range [0.10]: True

Outside range [-0.80]: True

Outside range [2.50]: True

Outside range [1.00]: False

Print id with width 2, name with width 10, min and max value with width 8 and precision 2.

Print number with width 8 and precision 2.

```
rangeChecker [ 5] 'target2 ' - 2.20 <= num <= 3.40
Outside range [ 0.10]: True
Outside range [ -0.80]: True
Outside range [ 2.50]: False
Outside range [ 1.00]: True
rangeChecker [ 6] 'target3 ' - -2.00 <= num <= -0.50
Outside range [ 0.10]: True
Outside range [ -0.80]: False
Outside range [ 2.50]: True
Outside range [ 1.00]: True
Try again? y
Range 0 Name, Min, Max: AgeRange, 45, 25
Traceback (most recent call last):
```

In this example, the min value is not smaller than the max value. The assert statement in `__init__()` will cause an `AssertionError`.

```
...
File "C:/Users/StudentBoss/Finalexam_Fall2022/PracticeFinalExam.py", line 116, in __init__
    assert max>min, f"Max ({max}) must be greater than min ({min})"
AssertionError: Max (25.0) must be greater than min (45.0)
```

END OF FINAL EXAM.

Remember, do not logout or turn off your computer when you are done.